Assignment 2: IA32 assembly programming

Due: 11:59PM Tuesday, November 11, 2025

Grader (TA): Rubaiya T Sholi <rsholi1@lsu.edu>

Note: You must submit this assignment in **two** places: 1) the classes server (used for running your code) and 2) Moodle (used for automatically detecting potential cheating). There will be no credit if you submit in only one place.

In this assignment, you will write a basic IA32 assembly routine using gnu assembly. First, create the directory **`prog2`**, which will be used for submission. Change to this directory, and issue the following command to get the files you need (Note: Do not ignore the period at the end of the command):

```
cp ~cs3501_lee/cs3501_F25/p2/* .
```

View and understand the provided `Makefile` and `main.c` so that you can write your own in the future.

In this assignment, you will write an **assembly program** in the provided file `prog2.s`. The file will implement a function with the prototype:

```
int prog2(int i, int j, int *k, int a[5], int *l);
```

This function will do three things:

1. Return `j - i + 3`
2. Set `*k = 10 * (*k);`
    - It is NOT allowed to use any multiplication or division instructions.
3. Set `*l = a[0] + a[1] + a[2] + a[3] + a[4];`
    - You are not required to use conditional jumping for this task, but using it is strongly recommended.
    - Note that, when you modify any callee-save register (`%ebx, %esi, or %edi`), you need to save and restore its old value.

Note: You are required to write a **comment** for **each instruction** to explain its purpose. You can write a comment by beginning with the "**#**" sign in your assembly code. In addition, you should include your **full name** and **LSU ID** number as a comment in your code.

I suggest you get this working in three distinct steps:

1. First, write a function taking the above arguments, but the only thing it does is return `j - i + 3`. You can test this function by running the provided `xtest` program. (Hint: the return value should be stored in `%eax`.)

2. Second, try to use the address passed in as the `k` parameter to change the caller's variable. You need to get the integer stored at the address, multiply it by 10 (without using any multiplication or division instruction), and then store the computed value at the address.
3. Third, try to use the address passed in as a parameter to read integers stored in the array. Note that the address is the starting address of the array, and you need to calculate an appropriate offset to read each array element.

⇨ You can hand in with '**`make submit`**'.

Here is an example run of my completed tester:

```
>make
gcc -Wall -g -m32 -c main.c
gcc -Wall -g -m32 -c prog2.s
gcc -Wall -g -m32 -o xtest main.o prog2.o

> ./xtest
j-i+3= 6
k*10= 100
array sum= 18

j-i+3= 9
k*10*10= 1000
array sum 2= 36
```

When you are ready to submit, you can do so with "**`make submit`**". Note that you can submit multiple times, but don't do so after the due date! Note that you will be graded using the provided `Makefile`, not your copy of it, so do not count on any changes you make to any file except `prog2.s`. **In addition, you must copy and paste** the content of your prog2.s into Moodle. This Moodle submission will be used to automatically detect potential cheating (https://grok.lsu.edu/article.aspx?articleId=20589).

Important notes:

- You should **NOT** send any assignment-related emails to the instructor. The TA has full control over the assignment grading process. The instructor will NOT reply to assignment-related emails and will NOT answer to assignment-related questions during his office hours. You can still ask high-level concepts.
- You should **NOT** send the TA (or the instructor) any emails enclosing your source code. In fairness to other students, the TA will only check your submitted file(s) after the assignment deadline. If you send any email enclosing your source code, there may be a penalty for your assignment grade. You can still ask high-level concepts.
- If there is no comment in your code, a grade of "zero" will be recorded.
- Your program will be compiled and tested only on the *classes* server for grading. You need to make sure that your code is running well on the server. In other words, even though your program is running well on another machine, that will not be considered during grading.

- Since this assignment is posted about two weeks before its deadline, last-minute requests for extension will not be granted. Being busy can never be a valid reason to request an extension when you have about two weeks for the assignment. The instructor/TA will not reply to such requests.
- It is very likely that last-minute questions sent on the due date will not be answered by TA. Note that you have about two weeks to work on this assignment.
- Lateness penalty: 20%, 40%, and 60% deduction submitted on the next three days after deadline respectively; a grade of zero if submitted later than those days.